ECE382M-7 Final Project Interim Report

TL-UL to AXI4-Lite Bridge

1. Basic Project Information

Project title: TL-UL to AXI4-Lite Bridge
Team name: Group 13
Team members: Yen-Yu Chen, Chia-Huang Chen, Amaury Jayr, Hao-Yu Chang

2. Project Summary

Different IP blocks in an SoC often use different bus protocols. Our project is a synthesizable Verilog bridge that lets a TileLink Uncached Lightweight (TL-UL) host access an AXI4-Lite peripheral. The bridge supports Get, PutFullData and PutPartialData. It returns AccessAckData or AccessAck with the original source, size and error status. Our goals are: (1) every request completes exactly once with the correct response; (2) both interfaces follow the valid/ready rules even when a channel stalls; (3) the design is synthesized and goes through APR, with area and timing reports. For the final demo we plan to show our simulation results and the final layout.

3. Literature Review

TileLink Spec v1.7.1 [1]. This is our main reference. Chapter 6 gave us the TL-UL messages and opcodes (Get = 4, PutFullData = 0, PutPartialData = 1, AccessAck = 0, AccessAckData = 1) and the rules for size, mask and address alignment. Chapter 4 says valid cannot depend on ready. It also says a receiver may hold ready low only for a few reasons, for example while it waits for a response. That is why our bridge handles one request at a time and keeps a_ready low while busy. Section 4.4 says a bridge to AXI should include a timeout. Our proposal did not have one, so we added it to our plan.

OpenTitan TL-UL documentation [2]. We used the same default widths (32-bit address and data, 8-bit source ID) and followed their protocol checker rules to detect illegal requests. For example, an error on a Get still returns AccessAckData with d_error = 1.

Chipyard TileLink/Diplomacy reference [3]. Rocket Chip converts TileLink to AXI with TLToAXI4 plus several extra blocks (fragmenter, deinterleaver and user yanker). TL-UL is single-beat and we allow only one outstanding request, so we do not need these blocks, which keeps our design small.

AMBA AXI specification [4]. In AXI, valid must stay high until the handshake, and the write response comes only after both AW and W are accepted. So we register each request before sending it to AXI and track AW and W separately.

4. Work Completed and Current Progress

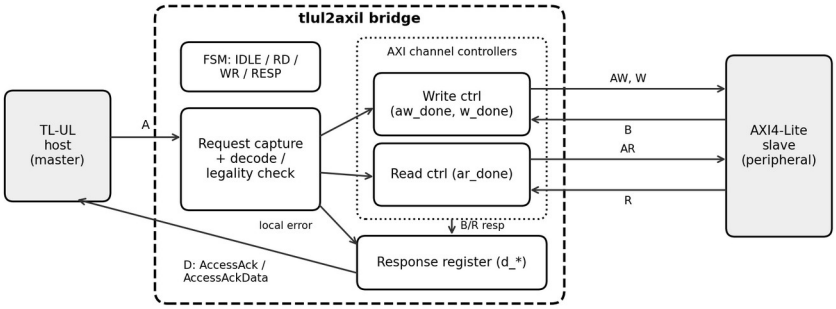


Figure 1. Block diagram of our bridge

The RTL is about 240 lines of Verilog and uses a 4-state FSM (IDLE, RD, WR, RESP). In IDLE the bridge accepts a request, checks whether it is legal, and saves the address, data, mask, source and size. A Get goes to RD, which sends AR and waits for R. A Put goes to WR, where AW and W can be accepted in any order; once both are done we wait for B. In RESP the D-channel outputs stay the same until d_ready is high. Illegal requests (bad opcode, size or mask, or a misaligned address) go directly to RESP with d_error = 1 and are never sent to AXI. An AXI SLVERR or DECERR response sets d_error.

For verification we wrote a testbench with four parts: a TL-UL host, an AXI4-Lite slave model with random ready signals, random delays and error address ranges, a scoreboard with a reference memory, and checkers that make sure valid and data stay stable during stalls. Table 1 shows the results.

Test	Result
Reset, full and partial writes, sub-word reads	Pass
AW first / W first / AW and W in the same cycle	Pass
Delayed B and R, D-channel backpressure	Pass
AXI SLVERR/DECERR and illegal TL requests	Pass
Random test, 40 seeds (about 4,500 requests each)	All pass

Table 1. Simulation results (Icarus Verilog)

Not done yet: running the tests with NC-Verilog on the course server, the timeout from Section 4.4 of the spec, synthesis, APR and timing analysis.

5. Team Responsibilities

Member	Done so far	Remaining work
Yen-Yu Chen	Bridge RTL (with Chia-Huang)	Finish the RTL (including the timeout) and make sure it runs with the course tools
Chia-Huang Chen	Bridge RTL (with Yen-Yu)	Finish the RTL (including the timeout) and make sure it runs with the course tools
Amaury Jayr	AXI4-Lite channel controllers	Study other TileLink/AXI functions and see what we can add
Hao-Yu Chang	Testbench and verification	Write TCL scripts for synthesis and APR, and run timing analysis

6. Remaining Work and Schedule

Date	Task	Owner
Nov 11	RTL runs on the course server (NC-Verilog)	Yen-Yu, Chia-Huang
Nov 14	Decide which extra functions to add	Amaury
Nov 17	Synthesis script and first area/timing report	Hao-Yu
Nov 20	Add timeout and extra functions, update tests	Yen-Yu, Chia-Huang, Amaury
Nov 23	First full APR run	Hao-Yu
Nov 25	Final RTL, post-APR timing check	All
Dec 1	Final presentation	All
Dec 9	Final report	All

Possible difficulties: our labs did not cover RTL synthesis, and the APR flow in Lab 2 was GUI-based without a clock, so writing the TCL scripts may take longer than expected. Thanksgiving break (Nov 26–29) is also right before the presentation.

7. Response to Teaching Team Comments

Comment: “Please assign individual task ownership for all members.” Response: Each member now has their own tasks (Section 5), and every item in the schedule has an owner (Section 6).

References

[1] SiFive, “TileLink Specification,” Version 1.7.1.
[2] OpenTitan, “TileLink-UL Bus,” https://opentitan.org/book/hw/ip/tlul/
[3] Chipyard Documentation, “TileLink and Diplomacy Reference,” https://chipyard.readthedocs.io
[4] Arm, “AMBA AXI and ACE Protocol Specification,” ARM IHI 0022.